

A Stochastic Dual-Timescale Model of Learning

From Motor Neurons to Language Models

Hima Shanthi Rayapati, Rui-Ling Lindsay Hon

May 5, 2026

Why Do We Learn at Two Different Speeds?

Crammer vs. Spaced Learner

- Crams night before exam
- Learns quickly, forgets within a week
- Studies in chunks over weeks
- Learns slowly, remembers for months

Same brain. Same material. Very different outcomes.

The core observation

The brain does not learn at a single speed. It runs **two processes simultaneously**:

- A **fast** process: learns quickly, forgets quickly
- A **slow** process: learns slowly, retains for much longer

This is true for motor learning, declarative memory, and , as we will show , artificial neural networks.

The Smith et al. (2006) Paper

Smith, Ghazizadeh, and Shadmehr (2006) formalized this in a **two-rate model of motor adaptation**.

Motor adaptation: how people adjust movements to correct for perturbations (e.g., a robotic arm that shifts your hand off-target).

Two behavioral observations that *no single-rate model can explain*:

Savings

When you relearn a skill after a break, you relearn it **faster than the first time**.
Something was retained across the gap, even though performance dropped.

Spontaneous Recovery

After learning to counteract one perturbation, then learning the opposite, the **original behavior partially reappears**. The first memory was suppressed, not erased.

Both require **at least two memory traces** operating at different timescales.

Where This Happens in the Brain

Fast Component , Cerebellum

- Detects movement errors quickly
- Drives rapid corrective adaptation
- Highly susceptible to interference
- New information easily overwrites old
- **Fast but fragile**

Slow Component , Motor Cortex + Striatum

- Changes gradually over many trials
- Holds learned patterns for much longer
- Resistant to interference
- **Slow but durable**

Complementary Learning Systems (CLS) theory

McClelland, McNaughton, O'Reilly (1995)

Brain region	Role	Speed
Hippocampus	Fast encoder	High
Neocortex	Slow consolidation	Low
Cerebellum	Fast motor adapt.	High
Motor cortex	Slow retention	Low

Mathematical consequence:

$$a_f > a_s \quad \text{and} \quad b_f > b_s$$

These constraints are biological facts, not arbitrary choices.

Why This Matters for AI

The same fast-slow tradeoff appears everywhere in machine learning.

Catastrophic Forgetting

A neural network trained on a new task forgets the old one almost immediately. This happens when the fast component dominates and the slow component is too weak. It is one of the biggest open problems in AI.

Gradient Descent with Momentum

- **Gradient** = fast component: reacts immediately to current error, noisy
- **Momentum** = slow component: running average of past gradients, stable

The question this project addresses

In large language models, **temperature** controls how much randomness goes into the output. Can we *predict from first principles* how output variability scales with temperature?

Answer: yes. It scales as σ^2 .

This is the central result of the stochastic extension.

From Discrete to Continuous Time

The original Smith et al. (2006) model is in **discrete trial time**:

$$f_{n+1} = A_f f_n + B_f e_n, \quad s_{n+1} = A_s s_n + B_s e_n$$

where e_n is the error on trial n .

Why reformulate in continuous time?

Continuous time gives us tools that do not exist in discrete time:

- Find equilibria in closed form
- Check stability via eigenvalues of the Jacobian
- Draw phase planes to visualize all possible trajectories
- Apply Itô calculus for the stochastic extension

This is not just a notational change. It opens the entire machinery of dynamical systems and SDE theory.

State Variables, Parameters, and Constraints

State variables (both $\in [0, 1]$, dimensionless):

- $f(t)$: fast learning component
- $s(t)$: slow learning component
- $x(t) = f(t) + s(t)$: total learning

Param	Value	Meaning
a_f	0.9	Fast decay rate
a_s	0.1	Slow decay rate
b_f	0.8	Fast learning rate
b_s	0.2	Slow learning rate

Biological constraints , always required

$$a_f > a_s \quad \text{and} \quad b_f > b_s$$

$a_f > a_s$: fast component forgets faster than slow. Cerebellum forgets more quickly than motor cortex. Violate this and the model is biologically meaningless.

$b_f > b_s$: fast component responds more strongly to error. If violated, the fast component cannot drive rapid early learning.

If either constraint fails, the model collapses to a single-timescale system and cannot reproduce savings or differential forgetting.

The Governing Equations

Each component is pulled by two opposing forces:

$$\frac{df}{dt} = \underbrace{-a_f f}_{\text{forgetting}} + \underbrace{b_f (T(t) - f - s)}_{\text{error-driven learning}} \quad (1)$$

$$\frac{ds}{dt} = \underbrace{-a_s s}_{\text{forgetting}} + \underbrace{b_s (T(t) - f - s)}_{\text{error-driven learning}} \quad (2)$$

Key features:

- $T(t) - f - s$ is the **error signal**: how far total learning is from target
- Both components respond to the **same shared error**

Intuition

- Fast: large b_f means strong response to error, but large a_f means it forgets quickly
- Slow: small b_s means weak response, but small a_s means it holds on

Equilibrium Analysis

For fixed target $T = 1$, set both derivatives to zero and solve:

$$f^* = \frac{b_f a_s}{\Delta}, \quad s^* = \frac{b_s a_f}{\Delta}, \quad \Delta = a_f a_s + a_f b_s + b_f a_s$$

Since all parameters are positive, $\Delta > 0$ always, so the equilibrium is **unique**.

Numerical values

$$f^* \approx 0.229, \quad s^* \approx 0.514, \quad x^* \approx 0.743$$

The slow component holds **most of the total learning** at equilibrium. This matches the biological picture: long-term retention is a cortical property.

What equilibrium means

Forgetting and learning are exactly balanced. The system is no longer changing. This is the long-run performance level if learning continues uninterrupted.

Stability Analysis

Does the system actually reach the equilibrium, or does it wander away?

For our linear system, the **Jacobian** is constant everywhere:

$$J = \begin{pmatrix} -(a_f + b_f) & -b_f \\ -b_s & -(a_s + b_s) \end{pmatrix}$$

Stability test for a 2×2 matrix:

$$\text{tr}(J) = -(a_f + b_f) - (a_s + b_s) < 0 \quad \checkmark \quad \det(J) = \Delta > 0 \quad \checkmark$$

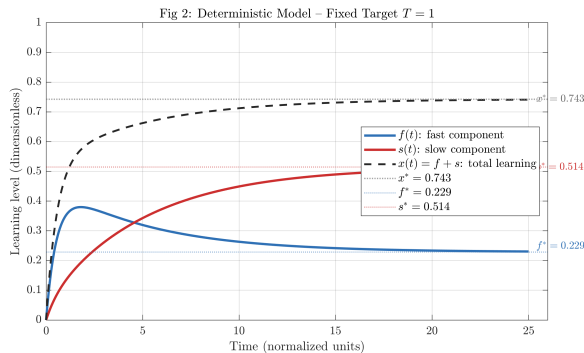
Both conditions hold for *any* positive parameter values. The equilibrium is **globally asymptotically stable**: every trajectory converges to (f^*, s^*) no matter where it starts.

The two eigenvalues encode the two timescales:

$$\lambda_1 \approx -0.194 \text{ (slow timescale)}, \quad \lambda_2 \approx -1.806 \text{ (fast timescale)}$$

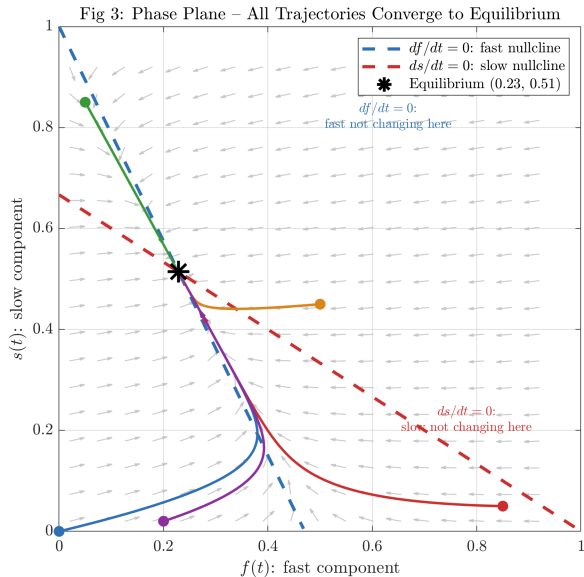
The ratio $|\lambda_2|/|\lambda_1| \approx 9.3$: the fast component converges roughly **nine times faster** than the slow one.

Deterministic Time Series



Fast (blue) rises steeply then settles at $f^* \approx 0.23$.
Slow (red) rises gradually and dominates at $s^* \approx 0.51$.

Phase Plane



All five trajectories from different starting points converge to the same equilibrium (star).

Nullclines (dashed) show where each component stops changing.

Why We Need Rest Periods

The basic deterministic model has a problem: **it never stops learning**.

$T = 1$ all the time means the system is always being driven toward its target. There are no breaks, no sleep, no task switches. This is not realistic.

Neuroscience:

- Sleep is not passive , it is active consolidation
- Hippocampus replays experiences during rest
- Fast (hippocampal/cerebellar) memory is fragile and resets
- Slow (cortical) memory consolidates and persists

AI:

- Rest = switching tasks or pausing training
- When a model switches to Task B, it stops receiving gradients from Task A
- Fast parameters reset almost immediately
- Slow parameters (momentum) retain a residual
- This is exactly why catastrophic forgetting happens

The Time-Varying Target

Replace the fixed $T = 1$ with a switching signal:

$$T(t) = \begin{cases} 1 & \text{during learning periods} \\ 0 & \text{during rest periods} \end{cases}$$

What changes mathematically:

The Jacobian J is **identical in both phases**.

The eigenvalues do not change. What changes is the equilibrium being approached:

Phase	Target equilibrium
Learning ($T = 1$)	$(f^*, s^*) \approx (0.23, 0.51)$
Rest ($T = 0$)	$(0, 0)$, complete forgetting

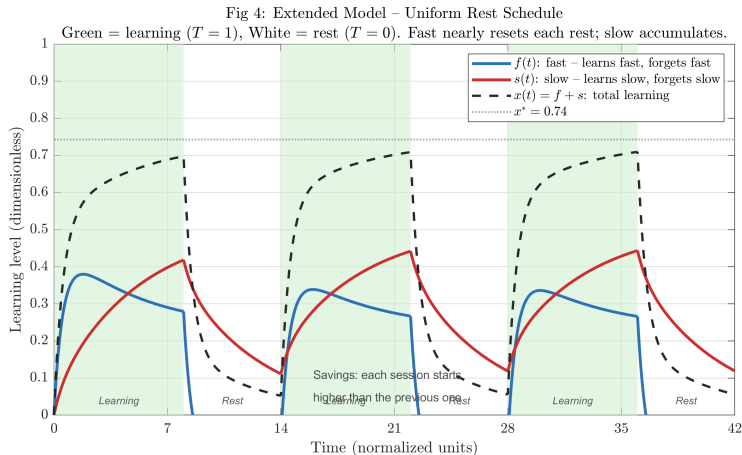
Why differential forgetting is automatic

During rest, the decay speed is still governed by $|\lambda_1|$ and $|\lambda_2|$. Since $|\lambda_2| \approx 9 \times |\lambda_1|$:

- Fast component collapses quickly toward 0
- Slow component barely moves

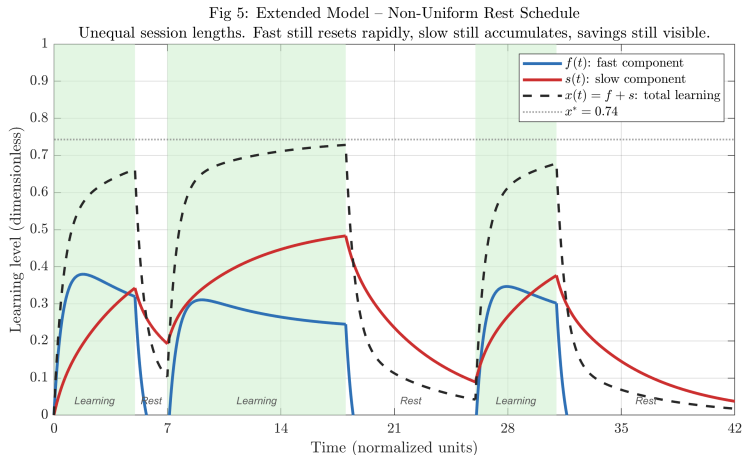
No new parameters needed. Differential forgetting falls out of the eigenvalue structure.

Uniform Rest Schedule



Green = learning, White = rest. Fast (blue) spikes during learning and nearly resets during rest. Slow (red) accumulates gradually and barely drops. The rising baseline of $x(t)$ across sessions is **savings**, relearning gets faster each time.

Non-Uniform Rest Schedule



Unequal session and rest lengths. The same qualitative behavior is preserved: fast still resets rapidly, slow still accumulates, savings still visible. The model's predictions do not depend on sessions being equally spaced.

What the Rest Period Extension Shows

Two key results emerge from the simulation, **without adding any new parameters**:

1. Savings

Total learning $x(t)$ at the start of each new session is *higher* than the start of the previous one. The system reaches its session peak faster each time. This is because $s(t)$ retained a residual across the rest period.

2. Differential Forgetting

$f(t)$ collapses almost to zero during rest while $s(t)$ barely moves. This mirrors what Smith et al. (2006) observed in real human motor experiments.

Both phenomena emerge from the eigenvalue structure alone.

$|\lambda_2| \gg |\lambda_1|$ is all it takes.

Why Add Noise?

Neuroscience:

- Ion channels open and close at random
- Synaptic vesicle release is probabilistic
- Same signal in, different output each time
- Intrinsic physical noise, not measurement error

Machine learning:

- SGD uses random mini-batches, not full data
- Every parameter update contains noise
- LLM temperature = randomness in output sampling

Central question

Can we predict from first principles how output variability scales with noise intensity σ ?

Noise source	Model
Ion channels	σdW
Mini-batch SGD	σdW
LLM temperature	σdW

All three: continuous, mean-zero, accumulating. Exactly what a Wiener process models.

What is a Wiener Process?

A Wiener process $W(t)$ is the mathematical model for pure accumulated randomness.

At every tiny moment in time, a tiny random number dW is added:

$$dW \sim \mathcal{N}(0, dt)$$

Mean zero, variance dt , independent of everything before it.

Three key properties:

- ① $\mathbb{E}[dW] = 0$ (equally likely up or down)
- ② $\mathbb{E}[dW^2] = dt$ (variance grows with time)
- ③ Independent increments (past tells you nothing)

Property 1 is what causes mean preservation.

Property 2 is what causes variance to grow.

Why not regular calculus?

In regular calculus, $(dt)^2 \approx 0$ so you ignore it. But since $dW^2 = dt$, squaring the noise gives something non-negligible. This forces us to use Itô calculus, which has an extra correction term accounting for this.

The Itô SDE System

We take the deterministic equations and append a noise term to each:

$$df = [-a_f f + b_f(T(t) - f - s)] dt + \sigma dW_f \quad (3)$$

$$ds = [-a_s s + b_s(T(t) - f - s)] dt + \sigma dW_s \quad (4)$$

Nothing in the drift changed. Same forgetting terms, same error-driven learning, same parameters. The noise rides on top of the existing dynamics.

In compact vector form, letting $\mathbf{X} = \begin{pmatrix} f \\ s \end{pmatrix}$:

$$d\mathbf{X} = \underbrace{(J\mathbf{X} + \mathbf{c}) dt}_{\text{same as before}} + \underbrace{\sigma d\mathbf{W}}_{\text{noise added}}$$

Same J as stability analysis

$$J = \begin{pmatrix} -(a_f + b_f) & -b_f \\ -b_s & -(a_s + b_s) \end{pmatrix}$$

Same σ for both components maps onto one controllable parameter (temperature).

Result 1: Mean Preservation - Setup

Question: If we run many noisy paths and average them, where does that average go?

Define the mean trajectory:

$$\mathbf{m}(t) = \mathbb{E}[\mathbf{X}(t)] = \begin{pmatrix} \mathbb{E}[f(t)] \\ \mathbb{E}[s(t)] \end{pmatrix}$$

Start from the SDE:

$$d\mathbf{X} = (J\mathbf{X} + \mathbf{c}) dt + \sigma d\mathbf{W}$$

Take expectations of both sides. We do this term by term:

Term	After expectation	Why
$d\mathbf{X}$	$d\mathbf{m}$	definition of $\mathbf{m}$
$(J\mathbf{X} + \mathbf{c}) dt$	$(J\mathbf{m} + \mathbf{c}) dt$	expectation is linear
$\sigma d\mathbf{W}$	$\mathbf{0}$	$\mathbb{E}[dW] = 0$ always

Result 1: Mean Preservation - Result

After taking expectations, the noise term vanishes and we are left with:

$$\frac{d\mathbf{m}}{dt} = J\mathbf{m}(t) + \mathbf{c}(t)$$

This is identical to the deterministic ODE.

The mean follows exactly the same path as the noiseless system, for any value of σ .

Why the noise vanished:

$\mathbb{E}[dW] = 0$ by definition of the Wiener process. At every moment the noise is equally likely to be positive or negative. So across many paths those kicks cancel out exactly.

This is not something we forced. It is a consequence of the noise structure.

What this means

Run 1000 noisy paths. Average them. The average will look like the deterministic solution, no matter how large σ is. Fig 6 confirms this: black mean line sits on top of red deterministic line at every noise level.

What is Stationarity?

Stationarity means variance has stopped growing and settled at a fixed value.

The bathtub analogy:

Imagine a bathtub with a tap and a drain.

- **Tap** = noise constantly injecting variance at rate σ^2
- **Drain** = restoring force pulling paths back toward the mean
- Drain rate grows as the water level rises

At first the tub fills fast, little water, little drainage.
As variance grows, the drain speeds up. Eventually
inflow = outflow and the level stops changing.
That stable level is P_∞ - the stationary variance.

Mathematically, stationarity means $\frac{dP}{dt} = 0$. We use this condition next to find P_∞ .

Timeline for our system

$t = 0$: All paths at zero. Variance = 0.

Early on: Noise spreads paths apart.
Variance grows.

Later: Restoring force pushes back.
Growth slows.

$t \rightarrow \infty$: Balance reached. Variance settles at P_∞ . This is stationarity.

Tracking Variance: The Covariance Matrix

Mean preservation tells us where things go *on average*. Now we want the *spread* around that average.

Define the covariance matrix $P(t)$, it tracks how spread out the paths are at each moment:

$$P(t) = \begin{pmatrix} \text{Var}(f) & \text{Cov}(f, s) \\ \text{Cov}(f, s) & \text{Var}(s) \end{pmatrix}$$

The diagonal entries $\text{Var}(f)$ and $\text{Var}(s)$ are what we care about most. They tell us how much each component individually spreads across runs.

What we want

We want P_∞ , the value $P(t)$ settles to at stationarity, and specifically how its diagonal entries depend on σ :

$$P_\infty = \begin{pmatrix} \text{Var}_\infty(f) & \cdot \\ \cdot & \text{Var}_\infty(s) \end{pmatrix}$$

The Lyapunov Equation

Using Itô's isometry, differentiating $P(t) = \mathbb{E}[(\mathbf{X} - \mathbf{m})(\mathbf{X} - \mathbf{m})^\top]$ gives:

$$\frac{dP}{dt} = JP + PJ^\top + \sigma^2 I$$

Term	What it does
$JP + PJ^\top$	Deterministic dynamics acting on the spread. Because J has negative eigenvalues, this term is negative – it pulls variance back down toward zero.
$\sigma^2 I$	Noise injection. At every instant, noise pumps variance into both components at rate σ^2 . The I means both components are hit independently with the same intensity.

These two forces fight each other. Noise pumps variance in. Dynamics pull it back. At stationarity they balance.

At Stationarity: Setting $dP/dt = 0$

At stationarity variance has stopped changing, so $\frac{dP}{dt} = 0$:

$$0 = JP_{\infty} + P_{\infty}J^{\top} + \sigma^2 I$$

Now make the substitution $P_{\infty} = \sigma^2 Q$:

Step 1: substitute in:

$$0 = J(\sigma^2 Q) + (\sigma^2 Q)J^{\top} + \sigma^2 I$$

Step 2: factor out σ^2 :

$$0 = \sigma^2 (JQ + QJ^{\top} + I)$$

Step 3: divide both sides by σ^2 :

$$\boxed{JQ + QJ^{\top} + I = 0}$$

No σ anywhere in this equation.

Q depends only on J , which depends only on a_f, a_s, b_f, b_s . Once you fix the biological parameters, Q is fixed forever regardless of noise level.

Solving for Q

Q is a 2×2 symmetric matrix: $Q = \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{12} & Q_{22} \end{pmatrix}$

Expanding $JQ + QJ^T + I = 0$ entry by entry with $J = \begin{pmatrix} -1.7 & -0.8 \\ -0.2 & -0.3 \end{pmatrix}$:

(1,1) entry: $2(-1.7)Q_{11} + 2(-0.8)Q_{12} + 1 = 0$

$$-3.4 Q_{11} - 1.6 Q_{12} = -1$$

(2,2) entry: $2(-0.3)Q_{22} + 2(-0.2)Q_{12} + 1 = 0$

$$-0.6 Q_{22} - 0.4 Q_{12} = -1$$

(1,2) entry: $(-1.7)Q_{12} + (-0.8)Q_{22} + (-0.2)Q_{11} + (-0.3)Q_{12} = 0$

$$-0.2 Q_{11} - 2.0 Q_{12} - 0.8 Q_{22} = 0$$

Solve this 3×3 linear system:

$Q_{11} \approx 0.771$	$Q_{12} \approx -1.014$	$Q_{22} \approx 2.343$
------------------------	-------------------------	------------------------

Result 2: The σ^2 Scaling Law

Since $P_\infty = \sigma^2 Q$:

$$\text{Var}_\infty(f) = \sigma^2 Q_{11} \approx 0.771 \sigma^2$$

$$\text{Var}_\infty(s) = \sigma^2 Q_{22} \approx 2.343 \sigma^2$$

What the scaling law says

- Variance grows **exactly as** σ^2
- Double $\sigma \Rightarrow$ variance increases by factor of **4**
- Halve $\sigma \Rightarrow$ variance decreases by factor of **4**
- Q_{11} and Q_{22} are fixed by model parameters, not by noise

Why the σ^2 Law Matters

Why eliminating σ from Q matters:

If Q depended on σ in a complicated way, the prediction would be messy and hard to test.

Because Q depends only on model parameters, the scaling is exact and testable with a simple log-log plot.

Falsifiable prediction

Plot $\log(\text{Var})$ vs. $\log(\sigma)$.

The slope should be close to **2.0**.

Fig. 7 shows slopes of 1.98 and 2.02.

Result 3: Why Slow Has More Variance

We found $Q_{22} \approx 2.343 > Q_{11} \approx 0.771$. So $\text{Var}_\infty(s) \approx 3 \times \text{Var}_\infty(f)$.

This might surprise people. The slow component is supposed to be the stable one. Why is it noisier?

Think of it as a spring being randomly pushed:

Fast component = stiff spring

Restoring force $|\lambda_2| \approx 1.81$.

Push it $\Rightarrow$ it snaps back fast.

Kicks get corrected before they accumulate.

Result: low stationary variance.

Slow component = loose spring

Restoring force $|\lambda_1| \approx 0.19$.

Push it $\Rightarrow$ it drifts slowly back.

Kicks linger for a long time.

Result: high stationary variance.

Both springs get pushed equally hard (σ^2 is the same). The difference is purely in how fast each one recovers.

Result 3: Why Slow Has More Variance

For a scalar system $dx = \lambda x dt + \sigma dW$ with $\lambda < 0$, stationary variance is:

$$\text{Var}_\infty(x) = \frac{\sigma^2}{2|\lambda|}$$

Larger $|\lambda|$ (stronger restoring force) $\Rightarrow$ smaller variance. Our coupled system is more complex, but the same logic holds:

Component	Eigenvalue	Restoring force	Stationary Var
Fast f	$\lambda_2 \approx -1.81$	Strong	$0.771 \sigma^2$ (low)
Slow s	$\lambda_1 \approx -0.19$	Weak	$2.343 \sigma^2$ (high)

Fast forgets noise. Slow accumulates noise.

Fast forgets learning quickly and also forgets noise quickly. Slow retains learning and also retains noise.

Result 3: Why Does This Matter?

The core consequence:

The slow component is your long-term memory, it survives rest periods and produces savings. But it is also the noisier one.

So long-term outcomes are less predictable than short-term ones.

What this means concretely:

Run the same learning protocol twice, same σ , same parameters:

- Fast component f ends up near the same value both times
- Slow component s can end up in **very different places**

Why does that spread matter?

Mean preservation says the *average* of many runs matches the deterministic solution. But in practice you never average many runs, you run the experiment once and get one outcome.

The paradox

The system that is supposed to be stable and permanent is actually the one that varies most across runs.

Result 3: Why Does This Matter? (Cont.)

For the **fast component**: one run lands close to f^* . Strong restoring force keeps correcting noise, so a single run is reliable.

For the **slow component**: one run can land far from s^* . Weak restoring force lets noise accumulate, so two identical training runs can end up with very different s values.

In the MNIST experiment: weight traces fan out across runs while the mean stays near the deterministic line, exactly what large Q_{22} predicts.

Conclusion

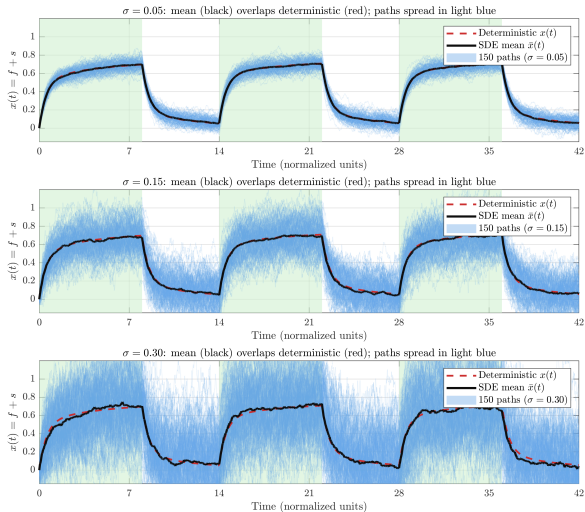
Noise does not just add randomness uniformly. It affects the two components very differently, and the component you rely on most for long-term retention is the one most affected.

This means in any noisy learning system, long-term memory is inherently less reproducible than short-term adaptation.

SDE Simulation

Fig 6: SDE Simulation at Three Noise Levels

Mean (black) tracks deterministic (red dashed) at every σ . Paths (light blue) spread more as σ grows.

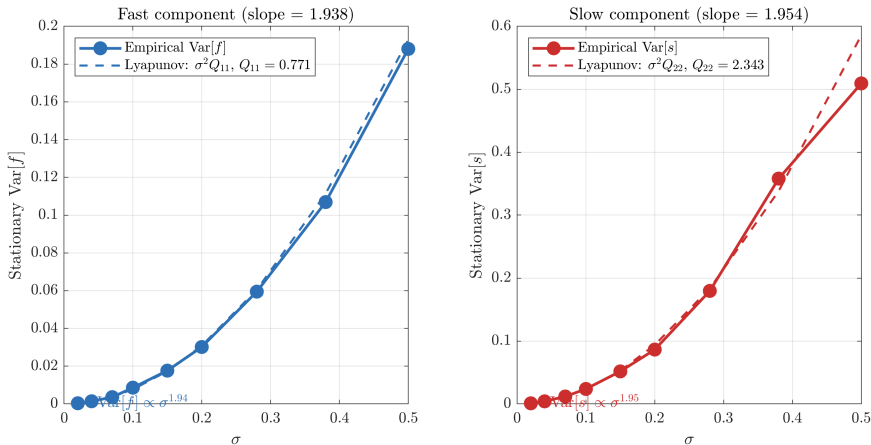


Three noise levels $\sigma \in \{0.05, 0.15, 0.30\}$.
Light blue lines = individual paths. Black = SDE mean. Red dashed = deterministic solution. Mean and deterministic overlap at every σ .

σ^2 Scaling Law , Numerical Verification

Fig 7: Verification of σ^2 Scaling Law

Dots = simulation. Dashed = Lyapunov analytical prediction. Slope ≈ 2.0 confirms $\text{Var} \propto \sigma^2$.

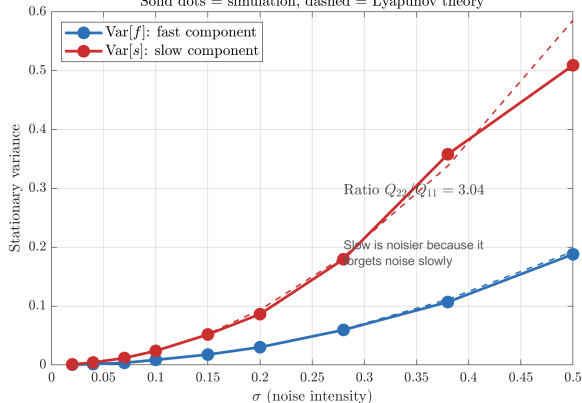


Log-log plot. Dots = simulation ($n = 300$ paths run to $t = 150$). Dashed = Lyapunov analytical prediction. Log-log slope ≈ 2.0 for both components confirms $\text{Var} \propto \sigma^2$.

Fast vs. Slow Stationary Variance

Fig 8: Counterintuitive Result – Slow Component Has Higher Stationary Variance

Solid dots = simulation, dashed = Lyapunov theory



At every σ , $\text{Var}[s]$ (red) $>$ $\text{Var}[f]$ (blue).
Ratio $Q_{22}/Q_{11} \approx 3.04$ is constant. Dots
= simulation, dashed = Lyapunov theory.
Both scale as σ^2 .

Three Results, Three AI Predictions

Mean preservation

Average LLM output across many runs at high temperature should match the zero-temperature output. If it doesn't, the noise is biased, a sign of a more complex model needed.

σ^2 scaling

Output embedding variance at temperature τ should scale as τ^2 . This is falsifiable: measure variance at three temperature values and check the slope on a log-log plot. Expected slope: 2.0.

Layer-wise variance

If early layers $\leftrightarrow$ fast component and late layers $\leftrightarrow$ slow component, then *late layers should show higher variance across runs* than early layers. This is measurable with any open-weight model.

Catastrophic forgetting

Fine-tuning on a new task = switching $T(t)$. Fast component resets quickly. Slow component retains a residual. Deliberate training pauses (rest phases) could preserve slow-component knowledge and reduce forgetting. Timescale: $1/|\lambda_2| \approx 0.55$ units for fast component to decay to e^{-1} .

Why Does the Fast Component Tend to Dominate in AI?

The fast component is easy to build

Every optimizer naturally wants to fix the current error immediately. That instinct, react strongly to each mistake, is the fast component. Without it the network barely learns. It is not a flaw, it is what makes learning happen at all.

The slow component requires deliberate friction

The slow component has to **resist** the current error signal. It says: “I see this mistake but I will not overreact. I will trust the accumulated direction over many steps.” That is computationally expensive and counterintuitive to build.

The stability-plasticity dilemma

- **Plasticity** = learn new things quickly (fast component)
- **Stability** = retain old things (slow component)

You cannot maximize both. Every choice that increases one decreases the other. This is the core reason catastrophic forgetting is so hard to solve.

Making the slow component more prominent (larger b_s , smaller a_s) means the system learns very slowly overall and takes much longer to reach equilibrium.

Why AI Struggles to Build a Genuine Slow Component

The AI problem

In LLMs, every weight gets the same gradient signal every step. There is no separation between fast and slow processing , just one noisy update repeated billions of times.

- Mini-batch gradients are noisy $\Rightarrow$ large $\sigma dW \Rightarrow$ fast component dominates
- Making progress requires large learning rate $\Rightarrow$ large b_f
- Momentum helps but forgets 10% of history every step , not truly slow

What the brain does differently

The brain runs **two physically separate systems** on genuinely different schedules:

- Hippocampus encodes single experiences immediately
- During **sleep**, it replays compressed summaries to the neocortex
- Neocortex builds stable representations over **weeks and months**
- The slow system never sees raw experience , only curated replays

What Our Model Explains

Core constraint

The constraints $a_f > a_s$ and $b_f > b_s$ create a fundamental tradeoff.

Strengthening the slow component , increasing b_s or decreasing a_s , shrinks the eigenvalue ratio $|\lambda_2|/|\lambda_1| \approx 9$.

- The two timescales get closer together
- Savings and differential forgetting weaken
- You cannot close the gap without losing the separation that makes the model work

The deeper gap

This model captures the mathematical tradeoff correctly. But in the brain, the slow component is driven by **replayed summaries**, not the raw error signal. That structural separation is what AI currently lacks , and what our SDE framework could be extended to model.

Interactive Parameter Exploration , R Shiny App

An R Shiny application lets you explore the model in real time.

Dual-scale Learning - Stochastic

Sliders: a_f , a_s , b_f , b_s , σ , number of paths

Three tabs:

- 1 **Deterministic + Rest:** $f(t)$, $s(t)$, $x(t)$ under the uniform schedule. Live equilibrium values and eigenvalues. Warning if biological constraints are violated.
- 2 **SDE Sample Paths:** Individual paths (light blue lines), SDE mean, deterministic reference. Shows mean preservation visually.
- 3 σ^2 **Scaling Law:** Log-log plot with fitted slope. Should be ≈ 2.0 .

Key things to try:

- Set $a_f \approx a_s$ or $b_f \approx b_s$, watch savings disappear
- Increase σ , variance grows, mean stays the same
- Decrease a_s toward zero , slow component becomes nearly permanent

Summary of Results

Deterministic model

- Unique, globally stable equilibrium at $(f^*, s^*) \approx (0.23, 0.51)$
- Two eigenvalues $\lambda_1 \approx -0.194$ (slow) and $\lambda_2 \approx -1.806$ (fast) with ratio ≈ 9
- Phase plane confirms global stability visually

Rest period extension

- Reproduces savings and differential forgetting without any new parameters
- Both emerge purely from $|\lambda_2| \gg |\lambda_1|$
- Robust to irregular session lengths

Stochastic extension (three results)

- ① **Mean preservation:** $\mathbb{E}[\mathbf{X}(t)]$ satisfies the same ODE as the deterministic system
- ② **σ^2 scaling:** $\text{Var}_\infty \propto \sigma^2$, verified analytically and numerically
- ③ **Slow has higher variance:** $Q_{22}/Q_{11} \approx 3.04$, fast forgets noise, slow accumulates it

From Theory to Experiment

What we derived analytically:

- A dual-timescale SDE with fast component $f(t)$ and slow component $s(t)$
- Three results: mean preservation, σ^2 scaling, slow has higher variance

The natural next question:

Do these equations actually work when applied to a real learning system?

We took the exact same SDE and implemented it as a neural network optimizer. Instead of abstract adaptation levels, f and s now update the actual weight parameters of a network trained on real data. The stochastic runs use $\sigma > 0$, the deterministic baseline uses $\sigma = 0$.

This is an empirical test of whether the math we derived describes real training dynamics.

Training a Neural Network with Stochastic Equations

Next to see the fast and slow learning stochastic equations in action, we trained a simple neural network on the MNIST training dataset. The dataset is a simple dataset with a bunch of handwritten single greyscale digits from 0 to 9.

We chose a relatively simple dataset to properly observe how parameters will wander during model training.



Figure: MNIST Dataset

Stochastic Dual Learning Rate Optimizer

The next step was to create an optimizer architecture combining the fast and slow state to update the model's learning parameter.

$$p_{t+1} = p_t + f_{t+1} + s_{t+1}$$

```
f += ( -group['af'] * f + group['bf'] * error) * dt +  
      group['sigma'] * torch.randn_like(f) * (dt**0.5)  
  
s += (-group['as_val'] * s + group['bs'] * error) * dt +  
      group['sigma'] * torch.randn_like(s) * (dt**0.5)  
  
p += f + s
```

We compare this to a deterministic model where we train the optimizer with $\sigma = 0$.

Training Process

During training, we found that we had to keep σ (noise) very low to ensure that it didn't consume the entire training process and override learning signals. There were other parameters needed major tuning during the training process to reach a more stable stochastic training result in terms of model loss, including slightly increasing learning rate, and reducing *as_val*, the temporal decay rate.

```
runs_det = [run_training_experiment(0.0, i) for i in range(3)]  
runs_stoch = [run_training_experiment(0.001, i) for i in range(3)]
```

Training Results

We can see the stochastic have a much more volatile loss. In general the higher noise can lead to better generalization during the LLM training process. Parameter also significantly shifts during the stochastic training process. This also means that the stochastic has a higher chance of finding a global minima for optimization over a local minima.

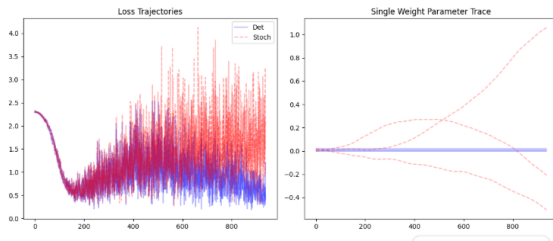


Figure: Stochastic (Red), Deterministic (Blue)

Injecting controlled noise is a great way to prevent models from "settling" at a local optimum too early allowing for generalization, and can prevent overfitting in some cases. This can especially useful for fitting models when you only have small, narrow datasets available for training. This can be in some ways comparable to extrapolations the human brain can make in the neurological learning process.